Image Inpainting and Object Removal

Week 5: Final Results, Runtime Analysis, and Failure Analysis

Ivan, Aleksandr, Egor, Tatiana | Computer Vision 2026 | November 3, 2026

Abstract

We compare classical, CNN-based, Transformer-based, and diffusion-based inpainting for zero-shot object removal on the CORNE-Val benchmark (218 samples, six mask conditions, 1,308 runs per model). This report consolidates the final quantitative results for all four paradigms, together with runtime measurements and a failure analysis. Pretrained LaMa is the most accurate method on every mask condition, reaching 19.75 dB region PSNR and 0.328 region LPIPS on effect-aware masks. ViTInpainter offers the best speed–quality trade-off on accurate masks. The two diffusion models we evaluated without task-specific object-removal training, FLUX.1-Fill and PixArt-α, are the slowest and are not competitive on reconstruction quality; we discuss why. This week we also prepared the final version of the web frontend that connects to the shared inference backend.

Contents

1 Project Status **2**

2 Final Quantitative Results **2**
- 2.1 Mask conditions . . . 2
- 2.2 Region metrics for all models . . . 2

3 Runtime and Computational Cost **3**

4 Failure Analysis **4**
- 4.1 Failure 1: semantic substitution by FLUX.1-Fill . . . 4
- 4.2 Failure 2: over-smoothed fill by PixArt-α . . . 4
- 4.3 Failure 3: hallucination at tight mask boundaries (LaMa and ViTInpainter) . . . 4
- 4.4 Failure 4: smearing on large coarse masks (all methods) . . . 5
- 4.5 Failure 5: Telea on structured backgrounds . . . 5

5 Frontend Integration **5**
- 5.1 Workflow . . . 5
- 5.2 Design decisions . . . 6

6 Discussion and Conclusions **7**

1 Project Status

The project consists of two parts that share one model interface, `inpaint(image, mask)`: a reproducible console benchmark on CORNE-Val (Evaluation Mode) and an interactive web application (Application Mode).

- **Benchmark.** All four paradigms (OpenCV Telea, pretrained LaMa, ViTInpainter, and a diffusion transformer) have been run on the full benchmark of $218 \times 6 = 1{,}308$ sample–mask pairs.
- **Frontend and backend.** This week we are preparing the final version of the frontend. The backend exposes a job-based REST API (`POST /api/v1/jobs`, `GET /api/v1/jobs/{id}`): the user uploads an image and draws a mask, selects models, and the interface polls the job status and shows each model's result in a carousel as soon as it is ready. Models are loaded lazily and cached, and LaMa weights are downloaded automatically on first use.
- **Diffusion models.** This week we continued testing diffusion-based approaches (FLUX.1-Fill and PixArt-α). Both are reported below; the conclusions about them are among the main findings of the project.

2 Final Quantitative Results

All metrics are computed against the clean ground-truth background `bg`. The primary metrics are measured on the masked region; full-image metrics are a secondary check. PSNR and SSIM are higher-is-better, LPIPS is lower-is-better. The best value in each block is in bold.

2.1 Mask conditions

Condition	Source	Operation
`sam_native`	`mask_sam`	exact object mask
`sam_dilated10`	`mask_sam`	dilation, radius 10
`sam_dilated20`	`mask_sam`	dilation, radius 20
`sam_bbox`	`mask_sam`	bounding box
`sam_ellipse`	`mask_sam`	enclosing ellipse
`eff_native`	`mask_eff`	object plus shadows and effects

Table 1: Mask conditions evaluated for every model.

2.2 Region metrics for all models

Key takeaway

Pretrained LaMa gives the best PSNR and LPIPS on all six mask conditions, and the best SSIM on five of six (Telea is marginally higher in SSIM on `eff_native`). Moving from a tight mask to an effect-aware mask improves every learned model: +3.81 dB for LaMa and +7.20 dB for ViTInpainter compared with `sam_native`.

Mask	Model	$PSNR_{reg}$ ↑	$SSIM_{reg}$ ↑	$LPIPS_{reg}$ ↓	Time, s ↓
sam_native	OpenCV Telea	12.74	0.603	0.487	**0.014**
	ViTInpainter	11.76	0.503	0.489	0.285
	PixArt-α	14.54	0.616	0.453	3.614
	LaMa	**15.94**	**0.657**	**0.359**	0.634
sam_dilated10	OpenCV Telea	14.65	0.664	0.446	**0.018**
	ViTInpainter	14.66	0.641	0.383	0.315
	PixArt-α	14.58	0.630	0.464	3.612
	LaMa	**16.83**	**0.670**	**0.357**	0.641
sam_dilated20	OpenCV Telea	15.26	0.671	0.453	**0.021**
	ViTInpainter	15.67	0.649	0.381	0.301
	PixArt-α	14.55	0.625	0.476	3.621
	LaMa	**17.53**	**0.672**	**0.358**	0.626
sam_bbox	OpenCV Telea	14.45	0.545	0.608	**0.025**
	ViTInpainter	14.20	0.466	0.534	0.299
	PixArt-α	14.51	0.543	0.556	3.597
	LaMa	**17.26**	**0.588**	**0.456**	0.636
sam_ellipse	OpenCV Telea	12.49	0.506	0.605	**0.022**
	ViTInpainter	11.34	0.404	0.581	0.298
	PixArt-α	12.71	0.517	0.562	3.621
	LaMa	**14.94**	**0.545**	**0.483**	0.645
eff_native	OpenCV Telea	17.42	**0.722**	0.420	**0.023**
	ViTInpainter	18.96	0.713	0.332	0.275
	PixArt-α	15.49	0.654	0.491	3.635
	LaMa	**19.75**	0.711	**0.328**	0.642

Table 2: Region metrics (means over 218 CORNE-Val samples). Standard deviations are given in the Week 2–4 reports; they are large (PSNR std of 4–6 dB) because the difficulty varies strongly between scenes.

3 Runtime and Computational Cost

Inference time is measured inside the model call only (image loading and saving are excluded). Telea runs on CPU; LaMa, ViTInpainter, and PixArt-α run on GPU.

Model	Device	Mean time, s	Relative to LaMa
OpenCV Telea	CPU	0.014–0.025	$\approx$ 30–45$\times$ faster
ViTInpainter	GPU	0.275–0.315	$\approx$ 2$\times$ faster
LaMa	GPU	0.626–0.645	1$\times$
PixArt-α	GPU	3.597–3.635	$\approx$ 5.6$\times$ slower
FLUX.1-Fill	GPU	tens of seconds per image	infeasible for the full benchmark

Table 3: Per-image inference time (range over the six mask conditions). The FLUX.1-Fill figure is a coarse observation from our memory-offloaded bfloat16 test runs, not a full benchmark measurement.

No model was trained or fine-tuned in this project, so there is no training time to report: all learned methods use public pretrained weights. FLUX.1-Fill (12B parameters) required memory offloading and bfloat16 precision and was still too slow to run across all 1,308 pairs within our hardware budget; this is why the full diffusion benchmark was done with the lighter PixArt-α ($\sim$0.6B parameters).

4 Failure Analysis

We inspected representative failures across all four paradigms. Each example below is a real output from the benchmark; the figures should be inserted from the saved qualitative comparison grids.

4.1 Failure 1: semantic substitution by FLUX.1-Fill

On an aerial shot of a fire truck on a gravel road, FLUX.1-Fill did not reconstruct the empty road; it generated a red sports car inside the mask.



Figure 1: Flux.1-Fill results example

Likely reason. FLUX.1-Fill is a text-guided generative model. Without an explicit background description it infers a plausible salient object from the mask silhouette and the road context, so it behaves as an object editor rather than a remover. We did not test a background prompt or negative prompt on the full set, so we cannot say how much a better prompt would help.

4.2 Failure 2: over-smoothed fill by PixArt-α

On a snow scene with a green sled (`eff_native`), PixArt-α removed the object but filled the hole with a hazy grey cloud that does not match the surrounding footprint texture. Its region PSNR on `eff_native` is 15.49 dB, below even classical Telea (17.42 dB) and well below LaMa (19.75 dB).

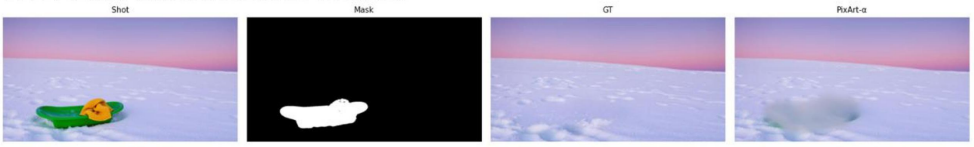


Figure 2: PixArt example

Likely reason. PixArt-XL-2-1024-MS is a text-to-image backbone used here through an inpainting pipeline, without fine-tuning for hole filling. It lacks the high-receptive-field perceptual and adversarial training that LaMa and ViTInpainter receive for this exact task, and the latent-space reconstruction produces low-frequency, washed-out textures.

4.3 Failure 3: hallucination at tight mask boundaries (LaMa and ViTInpainter)

With `sam_native` and moderately dilated masks, the cast shadow or boundary pixels of the object remain outside the mask. ViTInpainter then synthesizes dark structures along the shadow edge (11.76 dB on `sam_native`), and LaMa continues the shadow into the hole (15.94 dB, versus 19.75 dB with the effect-aware mask).

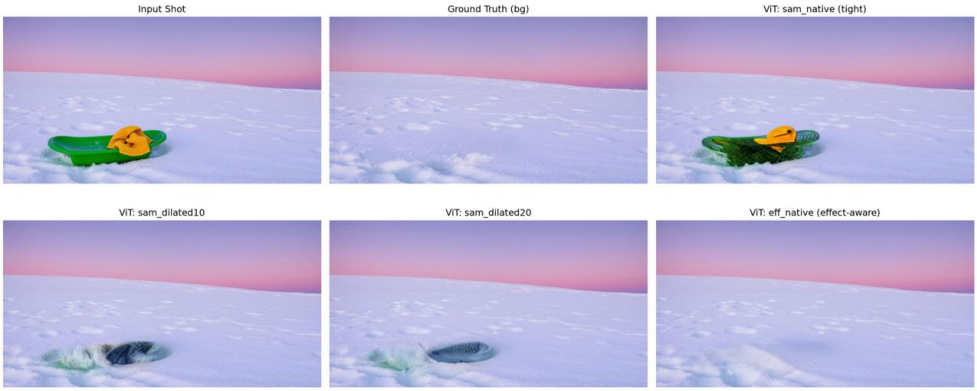


Figure 3:

Likely reason. Both networks treat the unmasked remnants as valid context and extend them inward. The error comes mainly from the mask, not from the network.

4.4 Failure 4: smearing on large coarse masks (all methods)

Replacing the object contour with a bounding box or an enclosing ellipse removes useful context. On `sam_ellipse`, region LPIPS worsens for all models: LaMa from 0.328 to 0.483, ViTInpainter from 0.332 to 0.581, Telea from 0.420 to 0.605. Feed-forward and classical methods produce blurry low-frequency patches in the middle of large holes because there are no nearby structural anchors.

4.5 Failure 5: Telea on structured backgrounds

Telea propagates boundary pixels inward without semantic priors, so it produces directional streaks and color puddles on complex objects and structured backgrounds (LPIPS up to 0.608 on `sam_bbox`).



Figure 4: Telea example

5 Frontend Integration

Application Mode connects a web interface to the same model wrappers that are used in the benchmark, so the demo and the evaluation run identical code.

5.1 Workflow

The user uploads a photo and draws a mask around the object to remove; the interface sends both to the backend together with the list of selected models. The backend creates a job and runs the models one after another. The frontend checks the job status periodically and shows each

model's output in a carousel as soon as it is ready; models that are still running show a loading placeholder, and a failed model shows its error without blocking the others.

Endpoint	Purpose
`GET /api/v1/models`	list of available models (`telea`, `lama`, `vit`, `flux`)
`POST /api/v1/jobs`	upload image, PNG mask, and model IDs; returns a job ID
`GET /api/v1/jobs/{id}`	job status and per-model results (status, output URL, runtime)
`GET /files/{id}/...`	stored input, mask, and model outputs

Table 4: Main endpoints of the inference API.

5.2 Design decisions

- **Shared interface.** The backend calls every model through the same `inpaint(image, mask)` method as the benchmark.
- **Mask validation.** The mask must be a grayscale PNG of exactly the same size as the image (white means remove). Invalid, mismatched, or empty masks are rejected with a structured error before any model runs.
- **Progressive results.** Each model has its own status (pending, running, succeeded, failed), and the job is marked partial if only some models succeed. A slow model does not delay the fast ones.
- **Resource handling.** Models are loaded lazily and cached, inference is serialized to share one GPU, and the LaMa weights are downloaded automatically on first use.
- **Mask dilation.** Our benchmark shows that tight masks cause boundary hallucinations (Section 4), so LaMa and ViTInpainter apply a small automatic dilation to user-drawn masks inside the application.

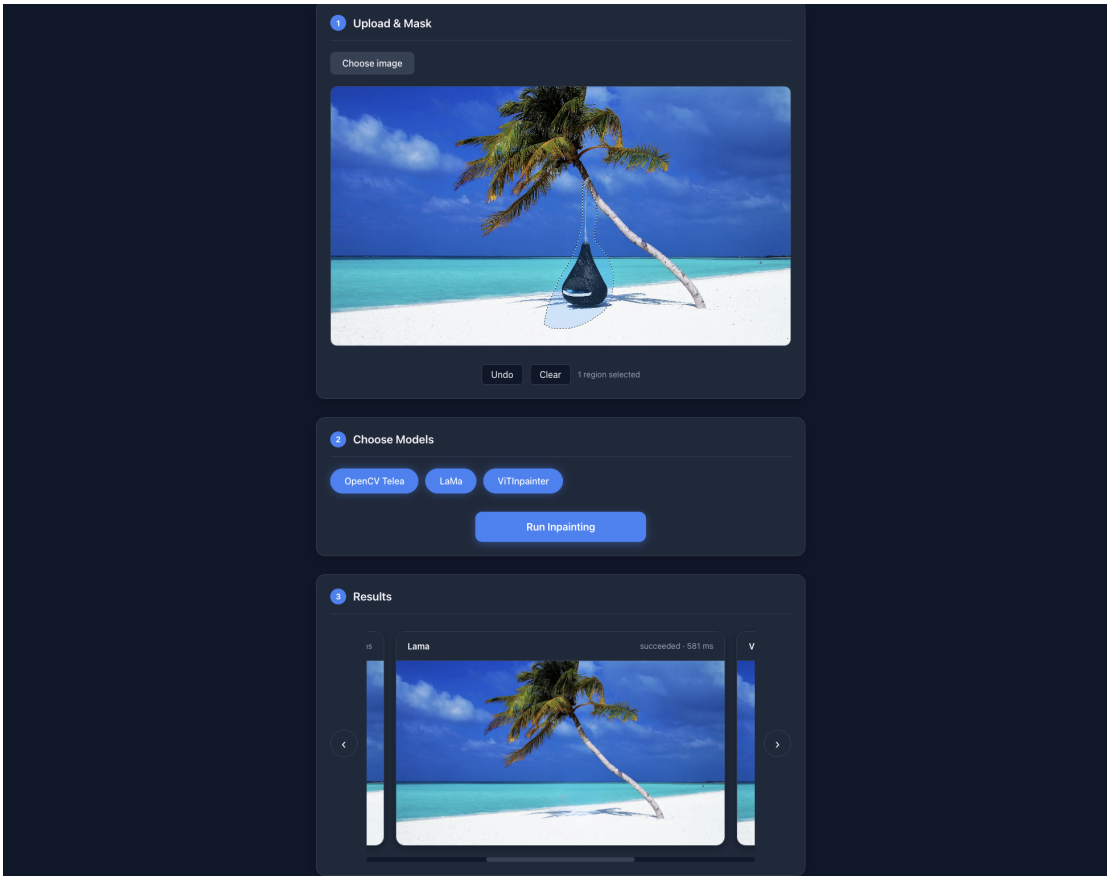


Figure 5: Frontend example

6 Discussion and Conclusions

Diffusion models without task-specific training

In our experiments, general-purpose diffusion models used without object-removal training were not a good choice for this task. FLUX.1-Fill replaced objects instead of removing them and was too slow for full evaluation; PixArt-α completed the benchmark but was $\sim$5.6$\times$ slower than LaMa and scored lower than classical Telea on effect-aware masks. Training or fine-tuning a diffusion model specifically for object removal would require paired removal data and GPU time well beyond our budget, so we did not attempt it.

- **Task-specific design matters more than size.** The best method, LaMa (Fast Fourier Convolutions), is much smaller and faster than the diffusion models and consistently more accurate.
- **Mask quality is the biggest practical factor.** Effect-aware masks improved every learned model, and tight masks caused most of the visible hallucinations. This suggests that automatic mask dilation is useful in the application.
- **Speed–quality trade-off.** ViTInpainter reaches 18.96 dB on effect-aware masks at 0.275 s, close to LaMa at half the time; Telea is the fastest but weakest on complex content.

Limitations. Our diffusion conclusions are evaluated zero-shot and without prompt tuning. A background prompt, negative prompting, or a model trained specifically for inpainting may change the results; we report this as an open question, not a refuted idea.

Next steps

Next week we finalize the project: complete the frontend and its integration with the backend, generate the final aggregated plots from `summary.csv`, finish the repository documentation, and prepare the final two-page technical report and the live demo. Code: https://github.com/Kazualov/bye-bye-AI.